# Shprehjet kushtore

## Programim në gjuhën C

Alion Emini

kurseinformatike.com

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Konteksti: nga Kapitulli 3

Në Kapitullin 3 pamë:

- **deklarata vendimmarrjeje**: `if`, `if ... else`, `if - else if - else`;
- strukturat ku zgjedhim *cilin bllok* kodi të ekzekutojmë;
- përdorimin e operatorëve logjikë dhe relacionalë.

# Çfarë do të shohim në këtë kapitull?

Në këtë kapitull do të shohim:

- **operatorin ternar** — zgjedh *një vlerë* brenda një shprehjeje;
- **makrot** dhe paraproçesorin e C-së;
- **vlerat Booleane** në C89 dhe C99;
- deklaratën `switch` si alternativë ndaj `if` të kaskaduara.

# Ideja bazë e operatorit kushtor

Ndonjëherë duam të zgjedhim **thjesht një vlerë** sipas një kushti:

- jo domosdoshmërisht të ekzekutojmë dy blloqe të ndryshme;
- por të vendosim *cilën vlerë* të përdorim në një shprehje;
- p.sh. cila vlerë t'i caktohet një variabli, çfarë të kthejë një `return`, çfarë të printojë `printf`.
- Për këtë C ofron **operatorin kushtor** (*ternar*) me simbolet `?` dhe `:`.

# Sintaksa e operatorit ternar

Forma e përgjithshme:

```
rezultati = kushti ? shprehjaTrue : shprehjaFalse;
```

Ku:

- `kushti` — shprehje logjike (0 = FALSE, jo-zero = TRUE);
- `shprehjaTrue` — përdoret kur kushti është i vërtetë;
- `shprehjaFalse` — përdoret kur kushti është i gabuar.
- lexohet: "Nëse kushti është i vërtetë, merre `shprehjaTrue`, përndryshe `shprehjaFalse`."

# Shembull: vlera më e madhe e dy numrave

```c
#include <stdio.h>

int main(void) {
    int i = 1, j = 2, k;

    /* Cakto vleren me te madhe midis i dhe j te k */
    k = (i > j) ? i : j;
    printf("k = %d\n", k); /* k = 2 */

    return 0;
}
```

# Shembull: vlera më e madhe e dy numrave

- `(i > j) ? i : j` kthen `i` nëse `i > j` është TRUE;
- përndryshe kthen `j`;
- në shembullin tonë `i > j` është FALSE, prandaj `k` merr vlerën `j`.

# Shembull: brenda një shprehjeje më të madhe

```c
#include <stdio.h>

int main(void) {
    int i = 1, j = 2, k;

    /* Nese i >= 0, merr i; perndryshe merr 0; pastaj shto j */
    k = (i >= 0 ? i : 0) + j;
    printf("k = %d\n", k); /* k = 3 */

    return 0;
}
```

# Shembull: brenda një shprehjeje më të madhe

Hapat:

- vlerësohet kushti `i >= 0`;
- nëse është TRUE operatori kthen `i`;
- nëse është FALSE kthen `0`;
- më pas rezultati i operatorit shtohet me `j`.

# Lidhja me `if ... else`

Shprehja:

```
rezultati = kushti ? shprehjaTrue : shprehjaFalse;
```

është logjikisht e barabartë me:

```
if (kushti) {
    rezultati = shprehjaTrue;
} else {
    rezultati = shprehjaFalse;
}
```

# Lidhja me `if ... else`

Diferenca kryesore:

- `if ... else` është **deklaratë** (statement);
- operatori ternar është **shprehje** — mund të përdoret kudo ku pritet një vlerë: `assign`, `return`, `printf`, ...

# Përdorimi në return

```
int max2(int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

```
int max2(int a, int b) {
    return (a > b) ? a : b;
}
```

# Përdorimi në `return`

- të dy versionet janë logjikisht të barabarta;
- i dyti është më kompakt dhe shpesh mjaft i lexueshëm;
- stil i përshtatshëm për *raste të thjeshta*.

# Përdorimi në printf

Versioni klasik:

```
if (i > j)
    printf("%d\n", i);
else
    printf("%d\n", j);
```

Version me operator ternar:

```
printf("%d\n", (i > j) ? i : j);
```

Ky stil është i dobishëm kur:

- zgjedh vetëm **një vlerë** për t'u shfaqur;
- logjika është shumë e thjeshtë;
- nuk ke nevojë për disa deklarata në secilin rast.

# Precedenca dhe lexueshmëria

## Precedenca dhe kllapat

- operatori `?:` ka **precedencë të ulët**;
- në shprehje të gjata është shumë e lehtë të ngatërrohet rendi i vlerësimit;
- rekomandohet:
  - përdor **kllapa** rreth operatorit ternar: `(kushti ? a : b)`;
  - shmang disa operatorë ternarë të mbivendosur në të njëjtën shprehje;
  - përdor `if ... else` sapo shprehja bëhet e vështirë për t'u lexuar.
- **qartësia** është më e rëndësishme se kursimi i 1–2 rreshtave.

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Paraproçesori dhe makrot

Para se të fillojë kompilimi, C kalon në një fazë **paraproçesimi**:

- trajtohen `#include`, `#define` dhe direktiva të tjera;
- zëvendësohen **tekstualisht** makrot;
- rezultati i kësaj faze i kalon më pas kompilatorit.

# Paraproçesori dhe makrot

## Përkufizim: Makro

Një **makro** është një emër që paraproçesori e zëvendëson me një tekst tjetër (vlerë, shprehje, fragment kodi) *para* kompilimit.

# Pse përdoren makrot?

Makrot mund të shërbejnë për:

- rritjen e **lexueshmërisë** së kodit;
- shmangien e **përsëritjes** së të njëjtit kod;
- centralizimin e **konstantave**;
- disa **optimizime të thjeshta** (sidomos historikisht, para funksioneve *inline*).

# Pse përdoren makrot?

Në përgjithësi dallojmë:

- **makro të thjeshta** (object-like);
- **makro me parametra** (function-like).

# Makro të thjeshta

```c
#include <stdio.h>

#define PI 3.14159
#define GREETING "Pershendetje, bote!"

int main(void) {
    printf("Vlera e PI: %f\n", PI);
    printf("%s\n", GREETING);
    return 0;
}
```

# Makro të thjeshta

- kudo që paraproçesori gjen `PI`, e zëvendëson me `3.14159`;
- kudo që gjen `GREETING`, e zëvendëson me `"Pershendetje, bote!"`.

# Makro vs const

## Makro vs const

Në C moderne, për konstanta numerike shpesh preferohet:

```
const double PI = 3.14159;
```

sepse:

- respekton rregullat e tipave (*type checking*);
- ka *scope* të parashikueshëm;
- është e dukshme për debugger-in.

Makrot megjithatë mbeten shumë të përdorura në:

- header-a;
- projekte ekzistuese;
- kompilim kushtor.

# Makro me parametra

```c
#include <stdio.h>

#define SQUARE(x) ((x) * (x))

int main(void) {
    int num = 4;
    printf("Katrori i %d eshte: %d\n", num, SQUARE(num));
    /* Output: Katrori i 4 eshte: 16 */
    return 0;
}
```

# Makro me parametra

- `SQUARE(x)` zëvendësohet me `((x) * (x));`
- përdorim kllapa:
  - rreth parametrave: `(x);`
  - rreth gjithë shprehjes: `((x) * (x));`
- kjo shmang surpriza me precedencën e operatorëve.

# Kujdes me efektet anësore

## Efektet anësore dhe makrot

Makrot *nuk* janë funksione:

- nuk kanë kontroll tipash;
- parametrat e tyre mund të vlerësohen disa herë.

```
SQUARE(i++) /* Mund te rrise i dy here! */
```

zëvendësohet me:

```
((i++) * (i++))
```

- `i` rritet dy herë;
- sjellje e papritur dhe e rrezikshme;
- për raste të tilla përdor një funksion normal ose ndaje llogaritjen në disa hapa.

# Makro MAX(a, b) me operatorin ternar

```c
#include <stdio.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main(void) {
    int x = 5, y = 10;

    printf("Max: %d\n", MAX(x, y)); /* Max: 10 */

    return 0;
}
```

# Makro `MAX(a, b)` me operatorin ternar

- kombinojmë makro me operatorin ternar;
- `MAX(a, b)` mund të shihet si një "funksion i vogël inline";
- sërish, **kllapat** janë kritike për të shmangur surpriza me precedencën;
- `MAX(i++, j++)` ka të njëjtin problem si `SQUARE(i++)`.

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Kompilim kushtor: pse na duhet?

## Qëllimi i kompilimit kushtor

Direktivat kushtore përdoren për:

- aktivizimin/çaktivizimin e kodit *debug*;
- zgjedhjen e implementimeve sipas platformës (Windows, Linux, . . . );
- veçori (*features*) opsionale;
- mbrojtjen e header-ëve nga përfshirja e shumëfishtë.

# #if, #elif, #else, #endif

Forma e përgjithshme:

```
#if KUSHTI
    /* kod nese KUSHTI eshte jo-zero */
#elif KUSHTI_TJETER
    /* kod nese KUSHTI_TJETER eshte jo-zero */
#else
    /* kod nese asnje prej kushteve siper s'eshte i vertete */
#endif
```

- kushtet janë **shprehje konstante integrale**;
- zakonisht bazohen te makrot e përkufizuara me `#define`.

# Shembull: kod *debug*

## Aktivizimi i kodit debug

```c
#define DEBUG 1 /* provo edhe 0 */

int main(void) {

#if DEBUG
    printf("Debug mode ON\n");
#endif

    printf("Programi kryesor...\n");
    return 0;
}
```

- nëse `DEBUG` është jo-zero, rreshti `printf("Debug mode ON\n");` përfshihet;
- nëse është 0, ky rresht *hiqet* krejtësisht para kompilimit;
- mund të përdorim edhe `defined(DEBUG)` në `#if`.

# #ifdef dhe #ifndef

- `#ifdef MAKRO` ⇒ nëse *MAKRO është përkufizuar*;
- `#ifndef MAKRO` ⇒ nëse *MAKRO nuk është përkufizuar*.

## Kod specifik për platforma

```
#ifdef _WIN32
    printf("Kodi po ekzekutohet ne Windows.\n");
#endif

#ifdef __linux__
    printf("Kodi po ekzekutohet ne Linux.\n");
#endif
```

# Header guard klasik

```c
/* my_header.h */

#ifndef MY_HEADER_H
#define MY_HEADER_H

void foo(void);
int bar(int x);

#endif /* MY_HEADER_H */
```

- herën e parë `MY_HEADER_H` nuk është përkufizuar, ndaj deklaratat përfshihen;
- më pas `MY_HEADER_H` është tashmë i përkufizuar, ndaj header-i anashkalohet;
- kështu shmangim gabimet e ri-deklarimit.

# `#if` vs `#ifdef` dhe gabime të shpeshta

- përdor `#ifdef`/`#ifndef` kur të intereson vetëm prezenca e makros;
- përdor `#if`/`#elif` kur ke nevojë për shprehje më të ndërlikuara (p.sh. `#if DEBUG && LEVEL > 1`).

# `#if` vs `#ifdef` dhe gabime të shpeshta

## Gabime të shpeshta

- harrimi i `#endif` në blloqe të gjata;
- qasje e tepruar në stilin `#if 0` për "komente" që harrohen për vite;
- logjikë shumë e komplikuar me direktiva kushtore që e bën kodin të pa-lexueshëm.

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Pse na duhen vlerat Booleane?

Programet kanë vazhdimisht nevojë për:

- variabla që përfaqësojnë një **kusht**;
- diçka që është **e vërtetë** ose **e gabuar**;
- p.sh. `finished`, `found`, `error`, ...

# Pse na duhen vlerat Booleane?

Në C89:

- nuk ekziston një tip `bool`;
- mbështetemi te rregulli: 0 = FALSE, çdo vlerë jo-zero = TRUE.

# 0 dhe jo-zero si vlera Booleane

## 0 dhe 1 si vlera Booleane në C89

- 0 përdoret si **FALSE**;
- çdo vlerë **jo-zero** përdoret si **TRUE**;
- zakonisht përdorim `int` për "flamuj" (flags).

```c
int flag;

flag = 0; /* FALSE */
/* ... me vone ... */
flag = 1; /* TRUE */
```

# Makro TRUE/FALSE

```c
#define TRUE 1
#define FALSE 0

int main(void) {
    int flag;

    flag = FALSE; /* me kuptim: flag eshte e gabuar */
    /* ... me vone ... */
    flag = TRUE; /* flag tani eshte e vertete */

    return 0;
}
```

- tani është e qartë që `flag` është variabël Boolean;
- vlerat e veta të vlefshme janë `TRUE`/`FALSE`.

# Testimi i flamujve Booleane

Dy stile të zakonshme:

```
/* Stil 1: krahason me makron TRUE */
if (flag == TRUE) {
    /* ... */
}

/* Stil 2: teston direkt vleren logjike te flag */
if (flag) {
    /* ... */
}
```

# Testimi i flamujve Booleane

```c
/* Stil 1: krahason me FALSE */
if (flag == FALSE) {
    /* ... */
}

/* Stil 2: perdor operatorin NOT */
if (!flag) {
    /* ... */
}
```

# Pse preferohet `if (flag)` / `if (!flag)`

## Stili i rekomanduar

- `if (flag)` dhe `if (!flag)`:
  - më të shkurtra dhe më të lexueshme;
  - funksionojnë saktë edhe nëse `flag` merr vlera të tjera jo-zero (2, 3, . . . ).
- `if (flag == TRUE)` është "e brishtë":
  - supozon që `flag` merr vetëm 0 ose 1;
  - për vlera të tjera jo-zero rezultati mund të mos jetë ai që pret lexuesi.

# Makro BOOL si "tip" Boolean

```c
#define TRUE 1
#define FALSE 0
#define BOOL int

int main(void) {
    BOOL flag; /* e njejte me: int flag; */

    flag = TRUE;
    if (flag) {
        /* ... */
    }

    return 0;
}
```

# Makro BOOL si "tip" Boolean

- për kompilatorin `BOOL` është thjesht `int`;
- për lexuesin, `BOOL flag;` tregon qartë që variabla është **Boolean**;
- mund të kombinohet me `TRUE/FALSE`.

# Tipi _Bool në C99

## Tipi _Bool

- është tip integral pa shenjë;
- mund të ruajë vetëm 0 (FALSE) ose 1 (TRUE);
- çdo vlerë tjetër jo-zero konvertohet automatikisht në 1.

# Shembull: _Bool në veprim

```c
#include <stdio.h>

int main(void) {
    _Bool flag;

    flag = 0; /* FALSE */
    printf("flag = %d\n", flag); /* shfaq 0 */

    flag = 1; /* TRUE */
    printf("flag = %d\n", flag); /* shfaq 1 */

    flag = 5; /* cdo vlere jo-zero -> behet 1 */
    printf("flag = %d\n", flag); /* shfaq 1 */

    if (flag) {
        printf("flag eshte TRUE\n");
    }

    return 0;
}
```

# <stdbool.h>: bool, true, false

Në vend që të përdorim direkt `_Bool`, C99 ofron header-in `<stdbool.h>`:

```
1 #include <stdbool.h>
```

i cili na jep:

- `bool` — alias për `_Bool`;
- `true` dhe `false` — korrespondojnë me `1` dhe `0`.

# Shembull: bool, true, false

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    bool flag; /* e njejte me: _Bool flag; */

    flag = false; /* e njejte me: flag = 0; */
    if (!flag) {
        printf("flag fillimisht eshte false\n");
    }

    flag = true; /* e njejte me: flag = 1; */
    if (flag) {
        printf("tani flag eshte true\n");
    }

    return 0;
}
```

# C89 vs C99 për vlerat Booleane

- **Në C89**:
  - nuk ka tip `bool`;
  - përdoret `int` + makrot `TRUE`/`FALSE` ose alias-i `BOOL`;
  - rregulli: `0` = FALSE, jo-zero = TRUE.
- **Në C99 dhe më pas**:
  - ekziston tipi `_Bool`;
  - `<stdbool.h>` na jep `bool`, `true`, `false`;
  - sintaksa bëhet më e pastër dhe më e uniformizuar me gjuhë të tjera.

# C89 vs C99 për vlerat Booleane

## Praktika e rekomanduar

Për programe moderne:

- përdor `#include <stdbool.h>`;
- shpall variablat logjike si `bool`;
- përdor `true`/`false` në vend të `1`/`0` kur ke të bësh me kushte.

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Nga if i kaskaduar te switch

Shembull nga Kapitulli 3:

```
if (nota == 4)
    printf("Shkelqyeshem");
else if (nota == 3)
    printf("Mire");
else if (nota == 2)
    printf("Mesatar");
else if (nota == 1)
    printf("Dobët");
else if (nota == 0)
    printf("Deshtim");
else
    printf("Note e paligjshme");
```

Kur rastet shtohen, ky stil bëhet më pak i lexueshëm.

# E njëjta logjikë me switch

```
switch (nota) {
    case 3:
        printf("Mire");
        break;
    case 2:
        printf("Mesatar");
        break;
    case 1:
        printf("Dobët");
        break;
    case 0:
        printf("Deshtim");
        break;
    default:
        printf("Note e paligjshme");
        break;
}
```

# E njëjta logjikë me `switch`

- më e qartë vizualisht;
- më e thjeshtë për t'u zgjeruar;
- në disa implementime mund të jetë edhe më efikase.

# Struktura bazë e switch

Forma tipike:

```
switch (shprehje) {
    case konstante1:
        /* deklarata per rastin 1 */
        break;

    case konstante2:
        /* deklarata per rastin 2 */
        break;

    /* ... raste te tjera ... */

    default:
        /* deklarata kur asnje rast nuk perputhet */
        break;
}
```

# Përbërësit kryesorë të `switch`

1. **Shprehja kontrolluese** brenda `switch (...)`: tip integral (`int`, `char`, `enum`, ...).
2. **Etiketat** `case case shprehje_konstante:`, ku shprehja është konstante integrale e njohur në kohë kompilimi.
3. **Deklaratat** ekzekutohen kur vlera e shprehjes përputhet me rastin.
4. **Klauzola** `default` (opsionale) trajton rastet kur asnjë case nuk përputhet.

# Çfarë *nuk* lejohet në `switch`

## Kufizime të `switch`

- shprehja kontrolluese **nuk** mund të jetë `float`/`double`;
- nuk mund të përdorim vargje (`char *`) si `case`;
- nuk lejohen dy `case` me vlerë të njëjtë;
- shprehja kontrolluese + `case`-t duhet të jenë integrale (ose `enum`).

# Grupimi i disa case-ve

```c
switch (nota) {
    case 4:
    case 3:
    case 2:
    case 1:
        printf("Kalues");
        break;

    case 0:
        printf("Deshtim");
        break;

    default:
        printf("Note e paligjshme");
        break;
}
```

- 4, 3, 2, 1 ndajnë të njëjtin bllok kodi;
- 0 trajtohet veçmas.

# Roli i break dhe *fall-through*

```c
switch (nota) {
    case 4:
        printf("Shkelqyeshem");
    case 3:
        printf("Mire");
    case 2:
        printf("Mesatar");
    case 1:
        printf("Dobët");
    case 0:
        printf("Deshtim");
    default:
        printf("Note e paligjshme");
}
```

# Roli i `break` dhe *fall-through*

- pa `break`, ekzekutimi "rrëshqet" në rastin tjetër (*fall-through*);
- nëse `nota == 3`, do të shfaqen të gjitha mesazhet nga `"Mire"` deri te `"Note e paligjshme"`;
- në shumicën e rasteve kjo *nuk* është ajo që duam.

# Fall-through i qëllimshëm

## Shembull me fall-through të qëllimshëm

```
switch (nota) {
    case 4:
    case 3:
    case 2:
    case 1:
        numri_kaluar++;
        /* Fall-through i qellimshem */
    case 0:
        numri_total++;
        break;
}
```

# Fall-through i qëllimshëm

## Shembull me fall-through të qëllimshëm

- për notat 1–4 rriten si `numri_kaluar` ashtu edhe `numri_total`;
- për notën 0 rritet vetëm `numri_total`;
- komenti ndihmon për të kuptuar që fall-through është i qëllimshëm.

# Shembull: data në formë "ligjore"

```c
#include <stdio.h>

int main(void) {
    int month, day, year;

    printf("Enter date (mm/dd/yy): ");
    scanf("%d/%d/%d", &month, &day, &year);

    printf("Dated this %d", day);
    switch (day) {
        case 1: case 21: case 31:
            printf("st"); break;
        case 2: case 22:
            printf("nd"); break;
        case 3: case 23:
            printf("rd"); break;
        default:
            printf("th"); break;
    }
```

# Shembull: data në formë "ligjore"

```
    printf(" day of ");
    switch (month) {
        case 1: printf("January"); break;
        case 2: printf("February"); break;
        case 3: printf("March"); break;
        case 4: printf("April"); break;
        case 5: printf("May"); break;
        case 6: printf("June"); break;
        case 7: printf("July"); break;
        case 8: printf("August"); break;
        case 9: printf("September"); break;
        case 10: printf("October"); break;
        case 11: printf("November"); break;
        case 12: printf("December"); break;
        default:
            printf("Invalid month");
            return 1;
    }

    printf(", 20%.2d.\n", year);
    return 0;
}
```

# Kur të përdorim `switch` dhe kur `if`

Përdor `switch` kur:

- ke një variabël integrale (`int`, `char`, `enum`);
- ke shumë raste të veçanta (0, 1, 2, 3, . . . );
- kushtet janë vetëm barazime (==) me konstante.

# Kur të përdorim `switch` dhe kur `if`

Përdor `if - else if - else` kur:

- kushtet përfshijnë intervale (<, >, <=, . . . );
- kombinon kushte me `&&`, `||`, `!`;
- ke `float`/`double` ose logjikë më të ndërlikuar.

# Përmbajtja


1. Shprehjet kushtore
2. Makrot në C
3. Direktivat kushtore
4. Bool në C89/C99
5. Deklarata `switch`
6. Mbyllje

# Çfarë mësove në këtë kapitull

Në këtë kapitull pamë:

- **operatorin ternar** `?:` për zgjedhjen e vlerave brenda shprehjeve;
- **makrot** dhe rolin e paraproçesorit: makro të thjeshta dhe me parametra;
- rreziqet me efektet anësore në makro dhe rëndësinë e kllapave;
- **vlerat Booleane** në C89 (`int`, `TRUE`/`FALSE`/`BOOL`) dhe në C99 (`_Bool`, `bool`, `true`, `false`);
- direktivat kushtore `#if`, `#ifdef`, `#ifndef`, `#endif` dhe *header guard*;
- deklaratën `switch`, rolin e `break` dhe konceptin e *fall-through*;
- kur është më e përshtatshme të përdorim `switch` dhe kur `if - else if - else`.

# Si lidhet me kapitullin 3 dhe çfarë vjen më pas

- Kapitulli 3 fokusohej te **strukturat e vendimmarrjes** në nivel deklaratash;
- ky kapitull shtoi:
  - vendimmarrje brenda *shprehjeve* (operatori ternar);
  - mjetet e paraproçesorit për konfigurim dhe lexueshmëri;
  - një mënyrë të pasur për të shprehur vlera Booleane;
  - një alternativë të pastër ndaj if-ve të gjata (`switch`).
- Në kapitullin tjetër do të kthehemi te **strukturat e përsëritjes** (`while`, `do ... while`, `for`) dhe do t'i kombinojmë me konceptet e këtij kapitulli.

# Faleminderit për vëmendjen!

Pyetje?